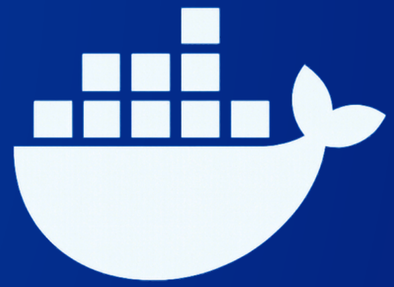


DOCKER

DOCKER: CONTAINERIZATION MADE SIMPLE

RAJ.

BEFORE DOCKER

BEFORE DOCKER, DEPLOYING APPLICATIONS ACROSS DIFFERENT ENVIRONMENTS WAS A NIGHTMARE. DIFFERENCES IN DEPENDENCIES, LIBRARY VERSIONS, AND OS CONFIGURATIONS LED TO THE INFAMOUS “WORKS ON MY MACHINE” PROBLEM.

WHAT IS DOCKER? (IN SIMPLE WORDS)

Docker is an open-source platform used to package, ship, and run applications inside lightweight, isolated environments called containers.

DOCKER'S SOLUTION:

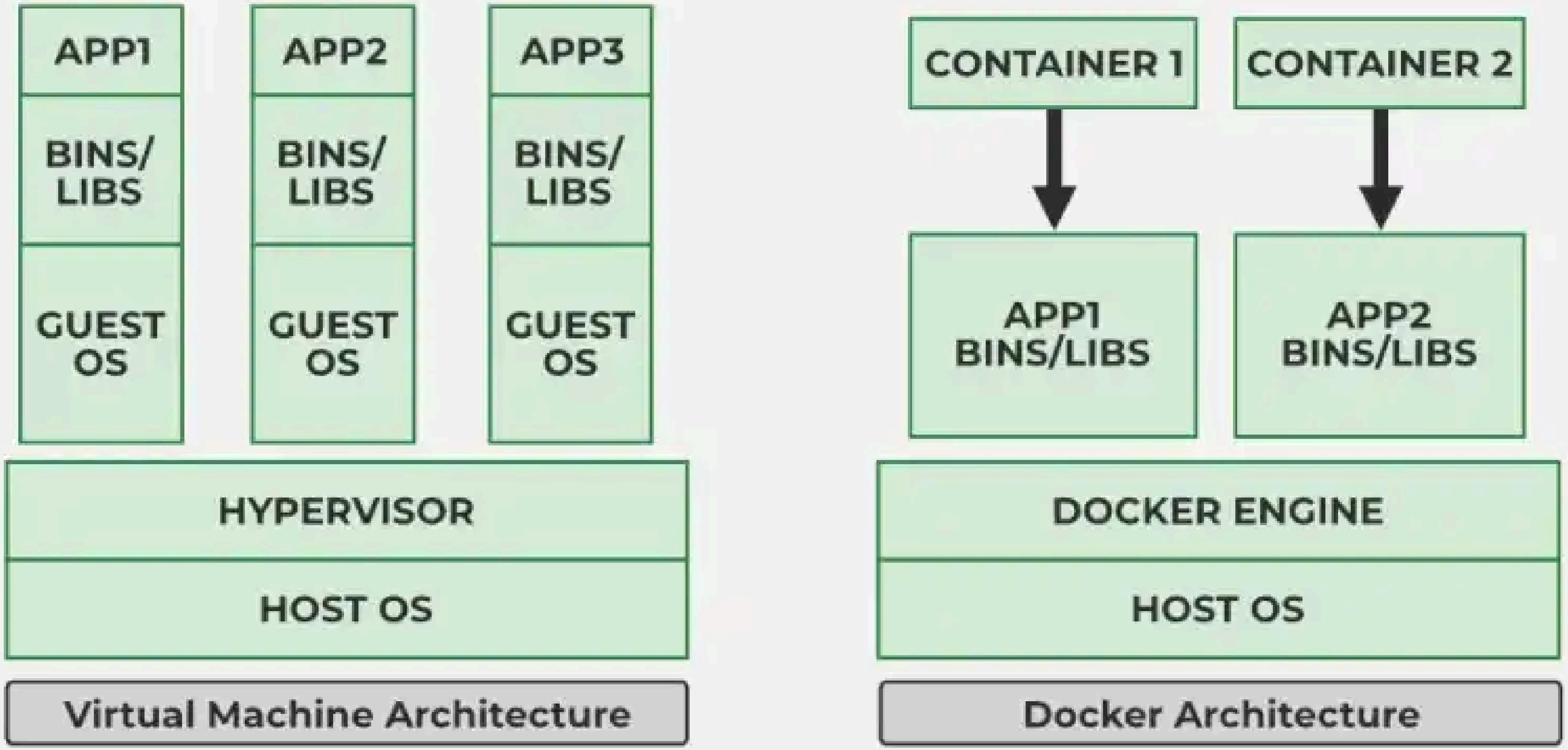
PORTABILITY: RUNS ANYWHERE IN LOCAL MACHINE, CLOUD, ON-PREM SERVERS.

CONSISTENCY: SAME BEHAVIOR IN DEVELOPMENT, TESTING, AND PRODUCTION.

LIGHTWEIGHT: NO FULL OS PER APP; CONTAINERS SHARE THE HOST KERNEL.

EFFICIENCY: STARTS IN SECONDS, USES FEWER SYSTEM RESOURCES.

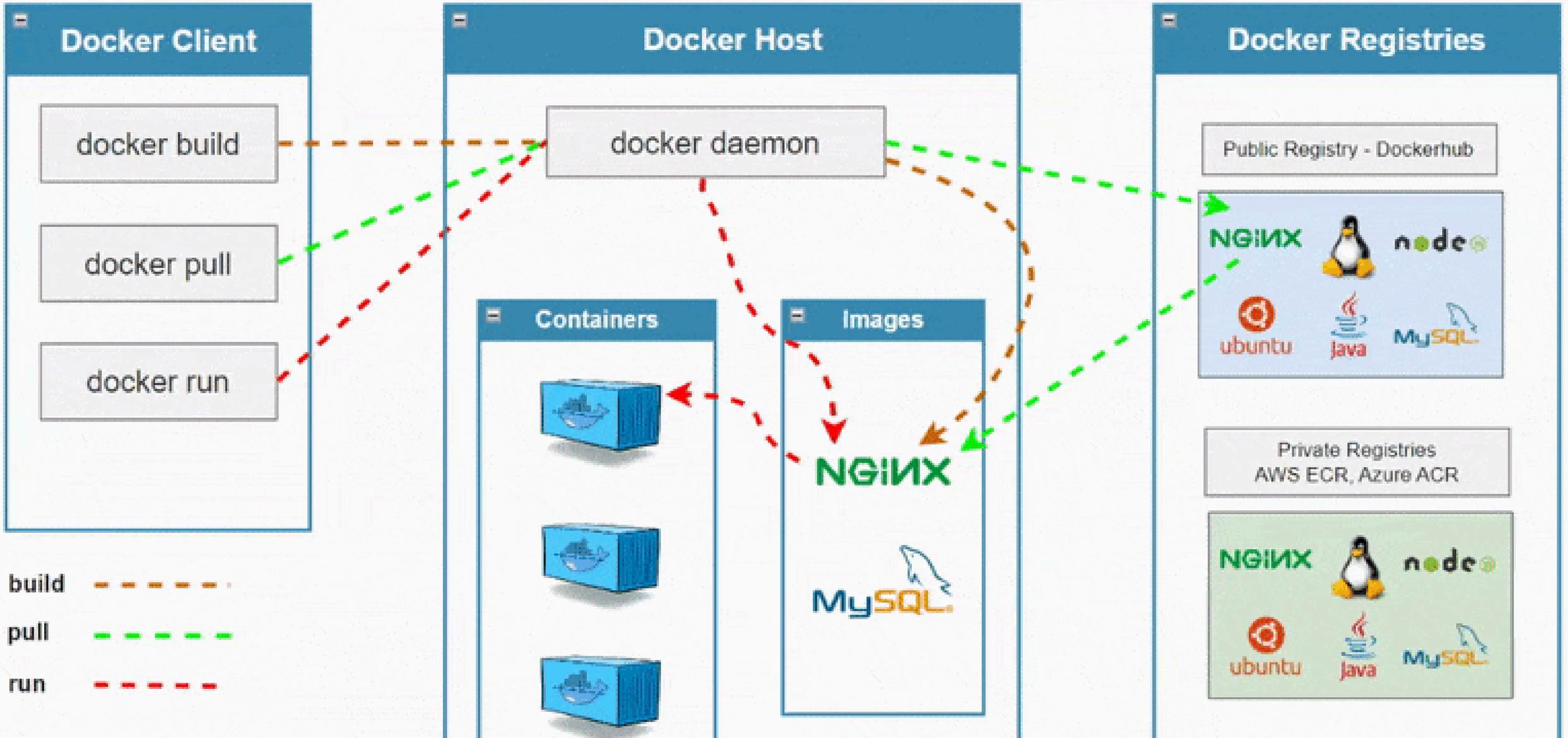
DOCKER VS VIRTUAL MACHINE



DOCKER VS VIRTUAL MACHINE

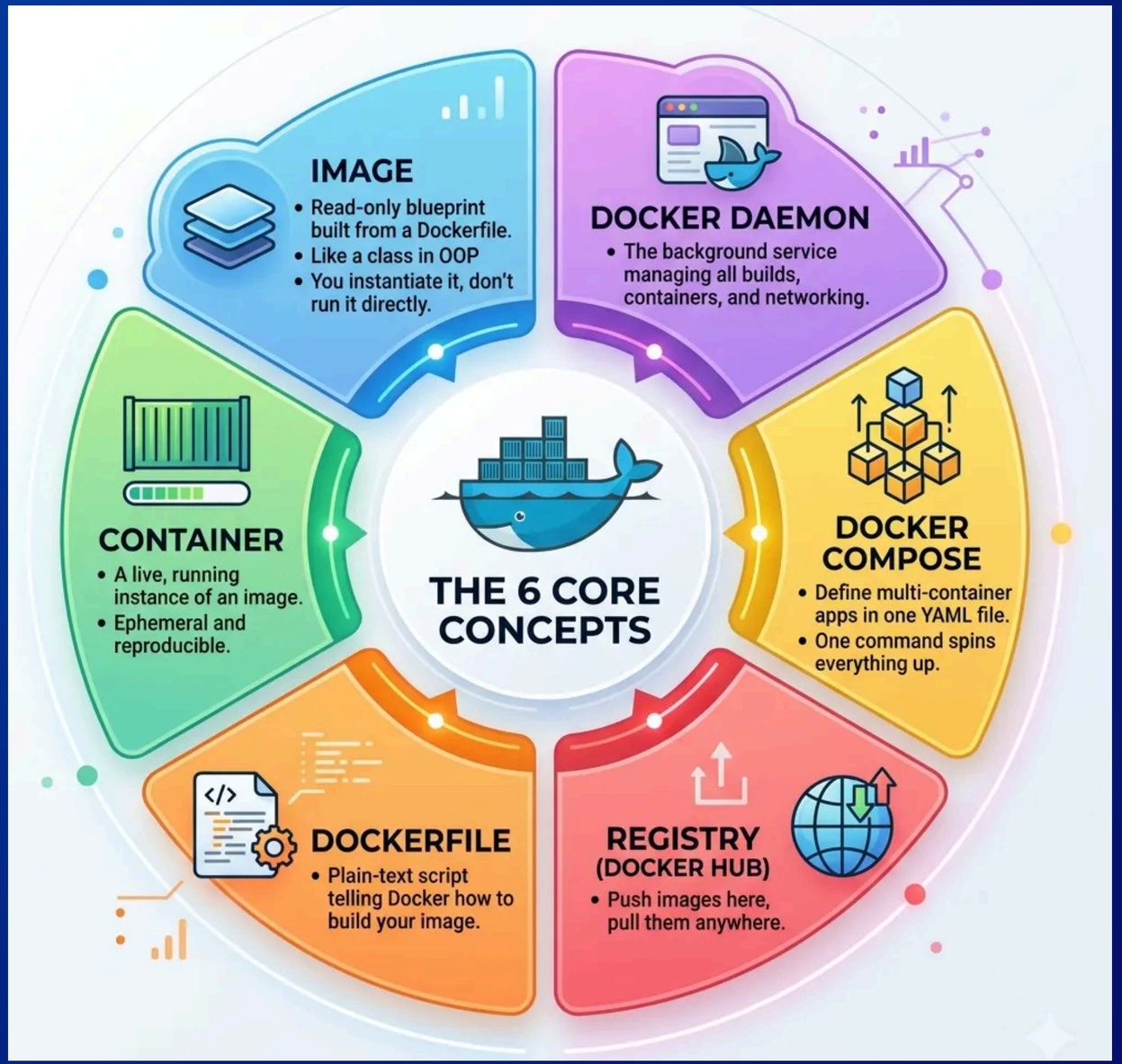
Feature	Docker Container	Virtual Machine
OS Usage	Shares the host OS kernel	Runs a complete separate OS
Size	Lightweight, usually MBs	Heavy, usually GBs
Startup Time	Starts in seconds	Takes minutes to start
Performance	Faster and efficient	Slower compared to containers
Resource Usage	Uses less CPU and RAM	Uses more CPU and RAM
Isolation	Process-level isolation	Full machine-level isolation
Portability	Easy to move across systems	Less portable because of full OS
Best Use Case	App deployment, microservices, CI/CD	Running different OS, full server setup
Example	One app running in a container	One full Ubuntu/Windows machine running as VM

Docker Architecture



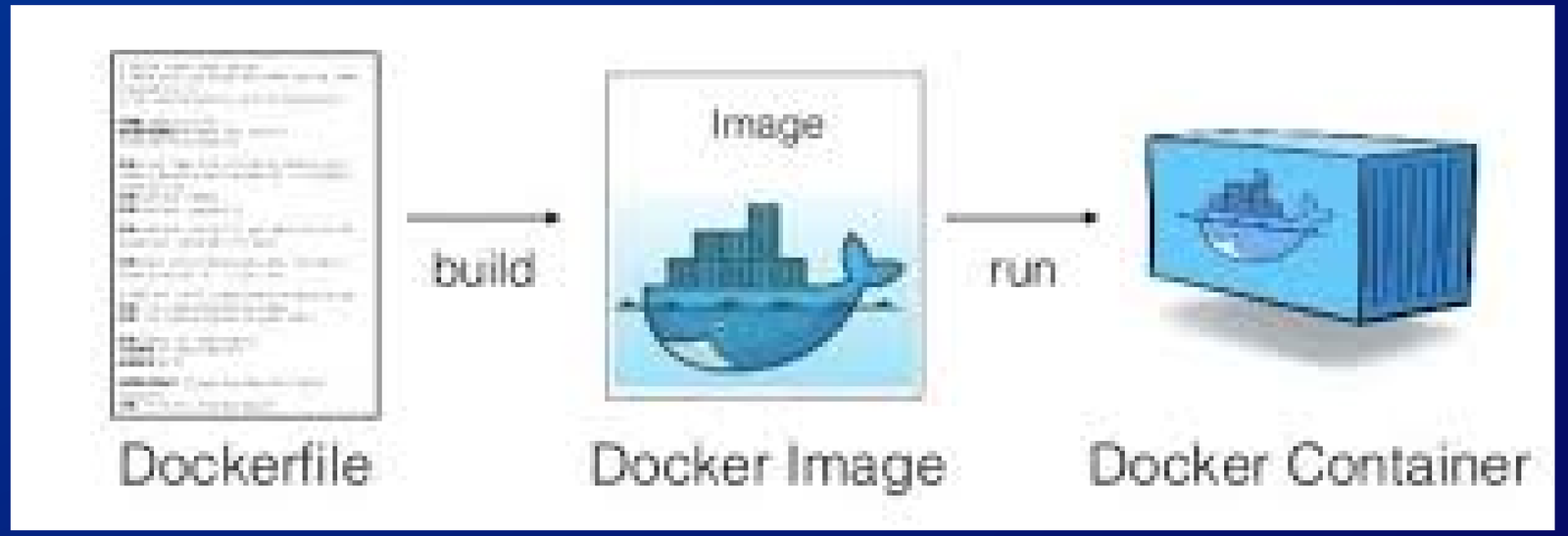
Component	Simple Explanation
Docker Client	Docker Client is the tool used to run Docker commands like docker build, docker run, and docker ps. It sends these commands to the Docker Daemon.
Docker Daemon	Docker Daemon runs in the background and does the main Docker work. It builds images, runs containers, and manages Docker resources.
Docker Images	Docker Image is a read-only template that contains the application code, libraries, dependencies, and runtime. Containers are created from images.
Docker Containers	Container is the running version of a Docker image. It runs the application in an isolated environment.
Docker Registry	Docker Registry is a storage place for Docker images. We can upload images to it or download images from it.
Docker Hub	Docker Hub is a public Docker Registry. It provides ready-made images like nginx, mysql, ubuntu, and many more.

THE CORE CONCEPTS



DOCKERFILE

THE DOCKERFILE USES DSL (DOMAIN SPECIFIC LANGUAGE) AND CONTAINS INSTRUCTIONS FOR GENERATING A DOCKER IMAGE. DOCKERFILE WILL DEFINE THE PROCESSES TO QUICKLY PRODUCE AN IMAGE. WHILE CREATING YOUR APPLICATION, YOU SHOULD CREATE A DOCKERFILE IN ORDER SINCE THE DOCKER DAEMON RUNS ALL OF THE INSTRUCTIONS FROM TOP TO BOTTOM.



DOCKERFILE EXAMPLE

```
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Line	Simple Explanation
FROM eclipse-temurin:17-jdk-alpine	This is the base image. It provides Java 17 to run our application.
WORKDIR /app	This creates and sets /app as the working folder inside the container.
COPY target/*.jar app.jar	This copies the Spring Boot .jar file from the target folder into the container.
EXPOSE 8080	This tells Docker that the application will run on port 8080.
ENTRYPOINT ["java", "-jar", "app.jar"]	This command starts the Java application inside the container.

```
docker build -t java-app .

docker run -p 8080:8080 java-app
```

Point	ENTRYPOINT	CMD
Meaning	Fixed main command of the container	Default command or default arguments
Use	Used when the container should always run one main command	Used when you want to give a default command that can be changed
Override	Not easily changed while running container	Easily changed while running container
Example	ENTRYPOINT ["java", "-jar", "app.jar"]	CMD ["--server.port=8080"]

Option	Meaning	Simple Explanation	Example
-t	Tag	Gives a name/tag to the Docker image	docker build -t java-app .
-d	Detached mode	Runs the container in the background	docker run -d java-app
-p	Port mapping	Connects your system port with container port	docker run -p 8080:8080 java-app
--name	Container name	Gives a custom name to the container	docker run --name java-container java-app
-e	Environment variable	Passes config values like DB username/passwor	docker run -e DB_USER=root java-app
-v	Volume	Saves data outside the container	docker run -v mydata:/app/data java-app
--rm	Auto remove	Deletes the container after it stops	docker run --rm java-app
-it	Interactive terminal	Opens terminal inside the container	docker run -it ubuntu bash

Multi-stage Dockerfile

File Edit View

```
# Stage 1: Build the application
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests


# Stage 2: Run the application
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Line	Explanation
FROM maven:3.9.6-eclipse-temurin-17 AS builder	First stage uses Maven and Java to build the app.
WORKDIR /app	Sets /app as the working folder inside the container.
COPY pom.xml .	Copies the Maven config file.
COPY src ./src	Copies the source code.
RUN mvn clean package -DskipTests	Builds the Java project and creates the .jar file.
FROM eclipse-temurin:17-jre-alpine	Second stage uses a small Java runtime image.
COPY --from=builder /app/target/*.jar app.jar	Copies only the final .jar file from the build stage.
EXPOSE 8080	Tells Docker the app runs on port 8080.
ENTRYPOINT ["java", "-jar", "app.jar"]	Starts the Java application.

Multi-stage Dockerfile is used to build the application in one stage and run it in a smaller final stage, which makes the Docker image lightweight, clean, and production-ready.

Docker Compose

Docker Compose is a tool used to run and manage multiple Docker containers together using one file called docker-compose.yml .

Service	Work
Java Backend	Runs Spring Boot application
MySQL Database	Stores application data
Docker Network	Connects backend and database
Volume	Saves database data permanently

Reason	Simple Explanation
Run multiple containers	We can run backend, database, frontend together.
One command deployment	We can start all containers using one command.
Easy configuration	Ports, environment variables, volumes, and networks are written in one YAML file.
Good for development	Developers can run the full project easily on their system.

Docker Compose Example

```yaml
version: "3.8"
services:
  # MySQL Database
  db:
    image: mysql:8.4
    container_name: mysql-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

  # Backend API
  backend:
    image: raj24kush/3-tier_app:latest
    container_name: backend
    restart: unless-stopped
    environment:
      DB_HOST: db
      DB_PORT: "3306"
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_NAME: ${DB_NAME}
      PORT: "8080"
    depends_on:
      db:
        condition: service_healthy
```

```yaml
  # Nginx Frontend
  nginx:
    image: nginx:alpine
    container_name: nginx
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - backend

volumes:
  mysql_data:
```

Ln 24, Col 1 | 1,245 characters | Plain text

Command	Simple Use
docker compose up -d	Starts all services in background.
docker compose up --build -d	Rebuilds images and starts all services.
docker compose ps	Shows running Compose containers.
docker compose logs	Shows logs of all services.
docker compose down	Stops and removes containers.

Docker Network

Docker Network is used to allow containers to communicate with each other and with the outside world.

Network Type	Simple Explanation
Bridge	Default network for containers on a single Docker host. Containers can talk to each other using container name.
Host	Container directly uses the host machine network. It does not create separate network isolation.
None	Container gets no network access. It is fully isolated from networking.
Overlay	Used when containers are running on multiple Docker hosts, mainly in Docker Swarm.
Macvlan	Gives container its own IP address like a real device on the physical network.

Example

- **docker network create my-network**
- **docker run -d --name mysql-db --network my-network mysql:8**
- **docker run -d --name java-app --network my-network -p 8080:8080 java-app**

Here, java-app can connect to MySQL using the name: mysql-db

Docker Volume

- Docker Volume is used to store container data permanently.
- By default, if a container is deleted, its data can also be lost.
- Volumes save the data outside the container, so data remains safe.

Reason	Simple Explanation
Data persistence	Data remains saved even after container delete/restart.
Database storage	Used to store MySQL, MongoDB, PostgreSQL data.
Easy backup	Volume data can be backed up easily.
Reuse data	Same volume can be used by another container.

Example: MySQL with Docker Volume

```
docker volume create mysql_data
```

```
docker run -d \
--name mysql-db \
-e MYSQL_ROOT_PASSWORD=root \
-e MYSQL_DATABASE=mydb \
-v mysql_data:/var/lib/mysql \
mysql:8
```

Docker Registry

Docker Registry is a storage place where Docker images are stored, uploaded, and downloaded.

Example:

When we run:

- **docker pull nginx**

Docker downloads the nginx image from a registry.

Command	Simple Use
docker login	Login to Docker Registry.
docker pull nginx	Download image from registry.
docker tag java-app username/java-app:latest	Add registry/repository name to image.
docker push username/java-app:latest	Upload image to registry.
docker logout	Logout from registry.

Docker Monitoring and Logs

Command	Simple Use
docker logs container-name	Shows logs of a container.
docker logs -f container-name	Shows live logs continuously.
docker logs --tail 50 container-name	Shows last 50 log lines.
docker compose logs	Shows logs of all Compose services.
docker compose logs -f java-app	Shows live logs of only Java app service.

Command	Simple Use
docker ps	Shows running containers.
docker ps -a	Shows all containers, running and stopped.
docker stats	Shows live CPU, memory, network, and disk usage.
docker inspect container-name	Shows detailed container information.
docker events	Shows real-time Docker events like start, stop, die.

Docker logs help us debug application issues, and Docker monitoring helps us check container health and resource usage like CPU and memory.

Command to Go Inside a Docker Container

docker exec -it container-name bash Opens bash shell inside a running container.

docker exec -it container-name sh Opens shell inside a running container, used when bash is not available.

Docker Scout Quickview Command

docker scout quickview image-name

Docker Scout Quickview is used to quickly check a Docker image for security issues and vulnerability summary.

DOCKER COMMANDS

Command	Simple Use
docker --version	Checks Docker version.
docker pull nginx	Downloads an image from Docker Hub.
docker images	Shows all Docker images on your
docker build -t my-app .	Builds a Docker image from Dockerfile.
docker run -d -p 8080:80 --name my-	Runs a container in background with port
docker ps	Shows running containers.
docker ps -a	Shows all containers, running and stopped.
docker logs my-container	Shows container logs.
docker exec -it my-container sh	Opens shell inside a running container.
docker stop my-container	Stops a running container.

Command	Simple Use
docker rm my-container	Removes a stopped container.
docker rmi my-app	Removes a Docker image.
docker compose up -d	Starts all services from Docker Compose.
docker compose down	Stops and removes Compose containers.
docker stats	Shows live CPU and memory usage of containers.

Command	Simple Use
docker compose up -d	Starts all services from docker-compose.yml in
docker compose up --build -d	Rebuilds images and starts all services.
docker compose ps	Shows running Compose containers.
docker compose logs -f	Shows live logs of all Compose services.
docker compose down	Stops and removes Compose containers.
docker network ls	Shows all Docker networks.
docker network create my-network	Creates a new Docker network.
docker network inspect my-network	Shows details of a Docker network.
docker volume ls	Shows all Docker volumes.
docker volume create mysql_data	Creates a new Docker volume.